

Lab 1.2 – Using Wireshark to Observe the TCP 3-Way Handshake View Network Traffic

Student Name: Mohamad Mostafa (ID: 30930)

Part 1: Prepare Wireshark to Capture Packets

Step 1: Retrieve your PC Interface Addresses

IPv4 Address: 192.168.1.116

MAC Address: 9C-AD-97-30-57-CF

```
Wireless LAN adapter Wi-Fi 4:

Connection-specific DNS Suffix  . : 
Description . . . . . : Ralink RT3290 802.11bgn Wi-Fi Adapter
Physical Address. . . . . : 9C-AD-97-30-57-CF
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80::698:5bd5:b97b:ddfc%28(Preferred)
IPv4 Address. . . . . : 192.168.1.116(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : Thursday, March 12, 2026 3:28:09 PM
Lease Expires . . . . . : Sunday, March 15, 2026 3:28:09 PM
Default Gateway . . . . . : 192.168.1.1
DHCP Server . . . . . : 192.168.1.1
DHCPv6 IAID . . . . . : 446475671
DHCPv6 Client DUID. . . . . : 00-01-00-01-25-9E-0C-79-5C-60-BA-B5-99-DA
DNS Servers . . . . . : 192.168.1.1
NetBIOS over Tcpi. . . . . : Enabled
```

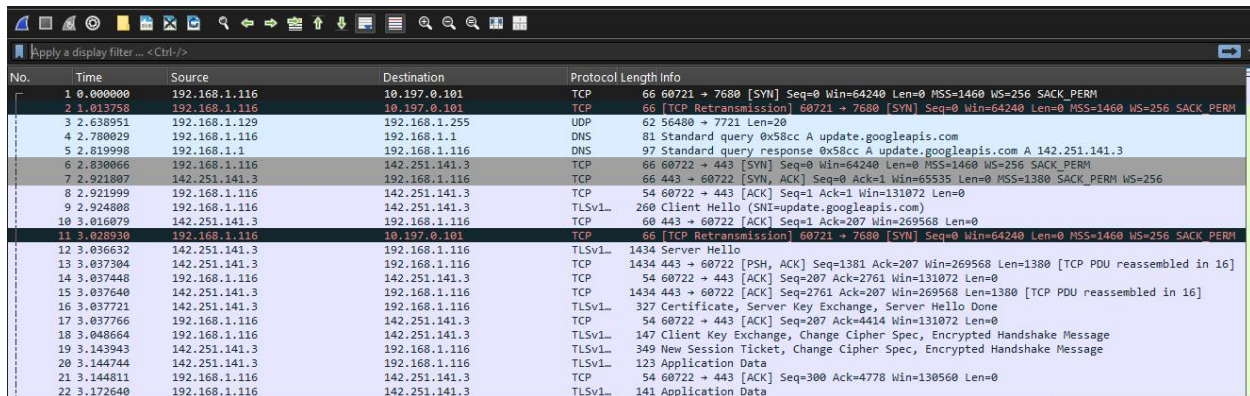
Step 2: Start Wireshark and Begin Capturing Data

Interface	Traffic	Link-layer Header	Promiscuous
Local Area Connection* 8		Ethernet	✓
Local Area Connection* 7		Ethernet	✓
Local Area Connection* 6		Ethernet	✓
▶ Wi-Fi 4		Ethernet	✓
▶ Local Area Connection* 12		Ethernet	✓
▶ Local Area Connection* 11		Ethernet	✓
▶ vEthernet (WSL)		Ethernet	✓
▶ Adapter for loopback traffic capture		BSD loopback	✓
▶ Ethernet 6		Ethernet	✓

Part 2: Capture, Locate and Examine Packets

Step 1: Capture the data

Opened <https://google.com> from the browser and stopped Capturing Data



The image shows a Wireshark packet capture of a web session. The packet list on the left shows 22 packets. The packet details pane on the right shows the selected packet (No. 10) as a TCP Reset (RST) from 192.168.1.116 to 10.192.0.101. The packet bytes pane shows the raw data of the reset packet.

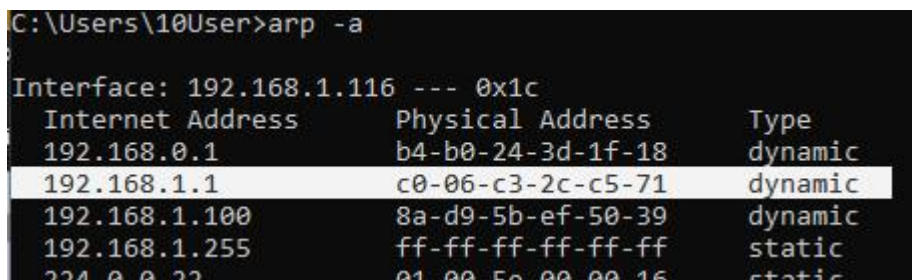
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.116	10.192.0.101	TCP	66	60721 → 7680 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2	1.013758	192.168.1.116	10.192.0.101	TCP	66	TCP Retransmission 60721 → 7680 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
3	2.638951	192.168.1.129	192.168.1.255	UDP	62	56480 → 7721 Len=20
4	2.780029	192.168.1.116	192.168.1.1	DNS	81	Standard query 0x58cc A update.googleapis.com
5	2.819998	192.168.1.1	192.168.1.116	DNS	97	Standard query response 0x58cc A update.googleapis.com A 142.251.141.3
6	2.830066	192.168.1.116	142.251.141.3	TCP	66	60722 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
7	2.921807	142.251.141.3	192.168.1.116	TCP	66	443 → 60722 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1380 SACK_PERM WS=256
8	2.921999	192.168.1.116	142.251.141.3	TCP	54	60722 → 443 [ACK] Seq=1 Ack=1 Win=131072 Len=0
9	2.924808	192.168.1.116	142.251.141.3	TLSv1	260	Client Hello (SNI=update.googleapis.com)
10	3.016079	142.251.141.3	192.168.1.116	TCP	60	443 → 60722 [ACK] Seq=1 Ack=207 Win=269568 Len=0
11	3.020920	192.168.1.116	10.192.0.101	TCP	66	TCP Retransmission 60721 → 7680 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
12	3.036632	142.251.141.3	192.168.1.116	TLSv1	1434	Server Hello
13	3.037304	142.251.141.3	192.168.1.116	TCP	1434	443 → 60722 [PSH, ACK] Seq=1381 Ack=207 Win=269568 Len=1380 [TCP PDU reassembled in 16]
14	3.037448	192.168.1.116	142.251.141.3	TCP	54	60722 → 443 [ACK] Seq=207 Ack=2761 Win=131072 Len=0
15	3.037640	142.251.141.3	192.168.1.116	TCP	1434	443 → 60722 [ACK] Seq=2761 Ack=207 Win=269568 Len=1380 [TCP PDU reassembled in 16]
16	3.037721	142.251.141.3	192.168.1.116	TLSv1	327	Certificate, Server Key Exchange, Server Hello Done
17	3.037766	192.168.1.116	142.251.141.3	TCP	54	60722 → 443 [ACK] Seq=207 Ack=4414 Win=131072 Len=0
18	3.048664	192.168.1.116	142.251.141.3	TLSv1	147	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
19	3.143943	142.251.141.3	192.168.1.116	TLSv1	349	New Session Ticket, Change Cipher Spec, Encrypted Handshake Message
20	3.144744	142.251.141.3	192.168.1.116	TLSv1	123	Application Data
21	3.144811	192.168.1.116	142.251.141.3	TCP	54	60722 → 443 [ACK] Seq=300 Ack=4778 Win=130560 Len=0
22	3.172640	192.168.1.116	142.251.141.3	TLSv1	141	Application Data

Step 2: Locate appropriate packets for the web session.

Before establishing connection: ARP, DNS, TCP 3-Way Handshake

(ARP : Resolves MAC address before IP communication)

The PC already had an ARP entry for the default gateway



```
C:\Users\10User>arp -a
```

Interface: 192.168.1.116 --- 0x1c	Internet Address	Physical Address	Type
	192.168.0.1	b4-b0-24-3d-1f-18	dynamic
	192.168.1.1	c0-06-c3-2c-c5-71	dynamic
	192.168.1.100	8a-d9-5b-ef-50-39	dynamic
	192.168.1.255	ff-ff-ff-ff-ff-ff	static
	224.0.0.22	01-00-5e-00-00-16	static

which means that it started with the DNS query to resolve www.google.com

(DNS: Resolves domain name (www.google.com) to IP address)

The PC must have the IP address of the DNS server, which is attempting to resolve the domain name www.google.com to the IP address of the web server. before it can send the first packet to the web server.

the IP address of the DNS server that the computer queried: 192.168.1.1

dns					
No.	Time	Source	Destination	Protocol	Length Info
4	2.780029	192.168.1.116	192.168.1.1	DNS	81 Standard query 0x58cc A update.googleapis.com
5	2.819998	192.168.1.1	192.168.1.116	DNS	97 Standard query response 0x58cc A update.google
39	3.452250	192.168.1.116	192.168.1.1	DNS	78 Standard query 0x39e9 A edgedl.me.gvt1.com
40	3.497668	192.168.1.1	192.168.1.116	DNS	94 Standard query response 0x39e9 A edgedl.me.gvt1.com
11278	15.121171	192.168.1.116	192.168.1.1	DNS	81 Standard query 0x4a72 A oneocsp.microsoft.com
11279	15.185837	192.168.1.1	192.168.1.116	DNS	166 Standard query response 0x4a72 A oneocsp.microsoft.com
16917	20.933996	192.168.1.116	192.168.1.1	DNS	70 Standard query 0x1f0e HTTPS google.com
16918	20.934380	192.168.1.116	192.168.1.1	DNS	70 Standard query 0x7148 A google.com
16982	20.989566	192.168.1.1	192.168.1.116	DNS	95 Standard query response 0x1f0e HTTPS google.com
16983	20.990048	192.168.1.1	192.168.1.116	DNS	86 Standard query response 0x7148 A google.com

The response from the DNS server. It contains the IP address of www.google.com.

142.250.203.206

95 Standard query response 0x1f0e HTTPS google.com HTTPS
86 Standard query response 0x7148 A google.com A 142.250.203.206

with nslookup command.

```
C:\Users\10User>nslookup google.com
Server: UnKnown
Address: 192.168.1.1

Non-authoritative answer:
Name: google.com
Addresses: 2a00:1450:4016:804::200e
          142.250.203.206
```

TCP three-way handshake

For establishing reliable connection

[SYN]

[SYN, ACK]

[ACK]

ip.addr == 142.250.203.206 && tcp					
No.	Time	Source	Destination	Protocol	Length Info
295	12.184233	192.168.1.116	142.250.203.206	TCP	66 60826 → 443 [SYN] Seq=0 W
312	12.279814	142.250.203.206	192.168.1.116	TCP	66 443 → 60826 [SYN, ACK] Se
313	12.279960	192.168.1.116	142.250.203.206	TCP	54 60826 → 443 [ACK] Seq=1 A

Step 3: Examine the information within packets including IP addresses, TCP port numbers, and TCP control flags

➤ [SYN]

Client requests connection, sends initial sequence number

No.	Time	Source	Destination	Protocol	Length	Info
295	12.184233	192.168.1.116	142.250.203.206	TCP	66	60826 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
Internet Protocol Version 4, Src: 192.168.1.116, Dst: 142.250.203.206						
Transmission Control Protocol, Src Port: 60826, Dst Port: 443, Seq: 0, Len: 0						
Source Port: 60826						
Destination Port: 443						
[Stream index: 6]						
[Stream Packet Number: 1]						
[Conversation completeness: Complete, WITH_DATA (63)]						
[TCP Segment Len: 0]						
Sequence Number: 0 (relative sequence number)						
Sequence Number (raw): 20814394						
[Next Sequence Number: 1 (relative sequence number)]						
Acknowledgment Number: 0						
Acknowledgment number (raw): 0						
1000 = Header Length: 32 bytes (8)						
Flags: 0x002 (SYN)						
000. = Reserved: Not set						
...0 = Accurate ECN: Not set						
...0 = Congestion Window Reduced: Not set						
...0 = ECN-Echo: Not set						
...0 = Urgent: Not set						
...0 = Acknowledgment: Not set						
...0 = Push: Not set						
...0 = Reset: Not set						
...1 = Syn: Set						
...0 = Fin: Not set						
[TCP Flags:S.]						

TCP source port number: 60826, it's a dynamic port assigned by the OS for client-side connections

TCP destination port number: 443, it's a static (well-known) port used by HTTPS

Flag "Syn" is set with relative number 1

....1 = Syn: Set

[TCP Flags:S.]

➤ [SYN,ACK]

Server acknowledges client request, sends its own sequence number

The image shows a Wireshark packet capture of a TCP connection. The top packet list shows three packets: a SYN from the client (312), a SYN-ACK from the server (312), and an ACK from the client (313). The selected packet (312) is a SYN-ACK from the server (192.168.1.116) to the client (142.250.203.206). The packet details show the following fields:

- Internet Protocol Version 4, Src: 142.250.203.206, Dst: 192.168.1.116
- Transmission Control Protocol, Src Port: 443, Dst Port: 60826, Seq: 0, Ack: 1, Len: 0
- Source Port: 443
- Destination Port: 60826
- [Stream Index: 6]
- [Stream Packet Number: 2]
- [Conversation completeness: Complete, WITH_DATA (63)]
- [TCP Segment Len: 0]
- Sequence Number: 0 (relative sequence number)
- Sequence Number (raw): 3881685182
- [Next Sequence Number: 1 (relative sequence number)]
- Acknowledgment Number: 1 (relative ack number)
- Acknowledgment number (raw): 20814395
- 1000 = Header Length: 32 bytes (8)
- Flags: 0x012 (SYN, ACK)
- 000. = Reserved: Not set
- ...0 = Accurate ECN: Not set
-0 = Congestion Window Reduced: Not set
-0 = ECN-Echo: Not set
-0 = Urgent: Not set
-1 = Acknowledgment: Set
-0 = Push: Not set
-0 = Reset: Not set
-1 = Syn: Set
-0 = Fin: Not set
- [TCP Flags:A..S..]

TCP source port number: 443

TCP destination port number: 60826

Flag “Syn” and “Acknowledgment” are set with relative number 1

....1 = Acknowledgment: Set

....1 = Syn: Set

[TCP Flags:A..S..]

➤ [ACK]

Client acknowledges server response

The image shows a Wireshark packet capture of a TCP connection. The top table lists three packets: a SYN from 192.168.1.116 to 142.250.203.206 (Seq=0), a SYN-ACK from 142.250.203.206 to 192.168.1.116 (Seq=0, Ack=1), and an ACK from 192.168.1.116 to 142.250.203.206 (Seq=1, Ack=1). The third packet is selected, and its details are shown below. The details pane shows the Internet Protocol Version 4, Transmission Control Protocol, and raw data. The TCP flags are shown as '.....A....'.

No.	Time	Source	Destination	Protocol	Length	Info
295	12.184233	192.168.1.116	142.250.203.206	TCP	66	60826 → 443 [SYN] Seq=0 Win=64240
312	12.279814	142.250.203.206	192.168.1.116	TCP	66	443 → 60826 [SYN, ACK] Seq=0 Ack=1
313	12.279960	192.168.1.116	142.250.203.206	TCP	54	60826 → 443 [ACK] Seq=1 Ack=1 Win=

Internet Protocol Version 4, Src: 192.168.1.116, Dst: 142.250.203.206

Transmission Control Protocol, Src Port: 60826, Dst Port: 443, Seq: 1, Ack: 1, Len: 0

Source Port: 60826

Destination Port: 443

[Stream index: 6]

[Stream Packet Number: 3]

[Conversation completeness: Complete, WITH_DATA (63)]

[TCP Segment Len: 0]

Sequence Number: 1 (relative sequence number)

Sequence Number (raw): 20814395

[Next Sequence Number: 1 (relative sequence number)]

Acknowledgment Number: 1 (relative ack number)

Acknowledgment number (raw): 3881685183

0101 = Header Length: 20 bytes (5)

Flags: 0x010 (ACK)

000. = Reserved: Not set

...0 = Accurate ECN: Not set

.... 0... = Congestion Window Reduced: Not set

.... .0.. = ECN-Echo: Not set

.... ..0. = Urgent: Not set

.... ...1 = Acknowledgment: Set

....0 = Push: Not set

....0 = Reset: Not set

....0. = Syn: Not set

....0 = Fin: Not set

[TCP Flags:A....]

Window: 512

Source Port: 60826

Destination Port: 443

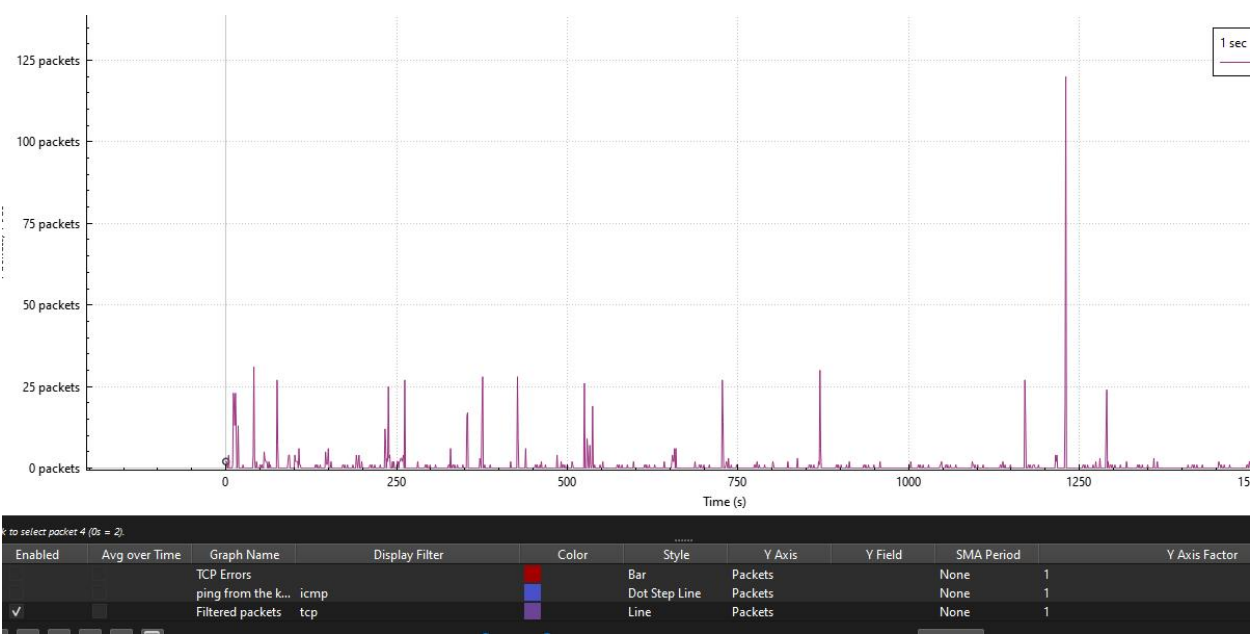
(Same source and destination as the second segment)

.... ...1 = Acknowledgment: Set

[TCP Flags:A....]

The TCP connection is established and communication between the source computer and the web server can begin.

TCP Graph Statistics (Packets/seconds)



three useful Wireshark filters for network administrators

`ip.addr == 192.168.1.1`

this filter would be useful to isolate all traffic to or from the default gateway

`!(arp or dns or tcp.port==443)`

Shows everything except ARP, DNS, and HTTPS traffic. Helpful for filtering out "noise" to see other types of traffic.

`tcp.flags.syn == 1`

Filters for TCP SYN packets. Helpful for identifying new connection attempts, which can be useful for detecting port scans or monitoring how many connections are being initiated.

other ways Wireshark could be used in a production network

Network Troubleshooting: When users report that an application is slow or a website is inaccessible, a network administrator can use Wireshark to capture traffic and analyze the TCP handshake, look for TCP retransmissions (which indicate packet loss), or verify that requests are reaching the server and replies are coming back.

Security Analysis: Wireshark can be used to inspect traffic for signs of malicious activity, such as unusual outbound connections (which might indicate malware), port scans, or clear-text transmission of sensitive data (like usernames and passwords in protocols like FTP or Telnet).

Application and Protocol Analysis: Developers and administrators can use Wireshark to see exactly how a custom or proprietary application communicates over the network, helping to identify inefficiencies or bugs in the application's network code.

Lab Completion Checklist

- ☑ Retrieved PC IP address and MAC address using ipconfig /all
- ☑ Started Wireshark and selected the correct network interface
- ☑ Successfully captured packets while visiting www.google.com
- ☑ Located DNS query and response packets
- ☑ Identified DNS server IP address and Google web server IP address
- ☑ Applied TCP filter to isolate TCP traffic
- ☑ Located and examined the first packet (SYN) of the three-way handshake
- ☑ Identified source port, destination port, and SYN flag in Packet 1
- ☑ Located and examined the second packet (SYN-ACK) of the three-way handshake
- ☑ Identified source port, destination port, SYN flag, and ACK flag in Packet 2
- ☑ Located and examined the third packet (ACK) of the three-way handshake
- ☑ Identified ACK flag in Packet 3
- ☑ Verified TCP connection was successfully established
- ☑ Listed three useful Wireshark filters for network administrators
- ☑ Described other ways Wireshark could be used in a production network

THANK YOU.

Mohamad Mostafa (ID: 30930)